

Encapsulation & Inheritance in C#

Encapsulation: Definition, Benefits & Validation

Encapsulation is a fundamental principle of object-oriented programming that bundles data (fields) and the methods that operate on that data into a single unit (class), while restricting direct access to some components.

Definition

Bundling data (fields) and methods into a single unit while restricting direct access to internal components

Benefits

- Protects object integrity by preventing invalid states
- Hides internal implementation details
- Allows internal changes without affecting external code

Validation

Logic inside accessors ensures data is correct before it is stored

```
public class BankAccount
{
    private decimal _balance; // hidden field

    public decimal Balance
    {
        get { return _balance; }
        private set
        {
            if (value < 0)
                throw new ArgumentException("Balance cannot be negative.");
            _balance = value;
        }
    }

    public void Deposit(decimal amount)
    {
        if (amount <= 0)
            throw new ArgumentException("Deposit must be positive.");
        Balance += amount; // uses validation in setter
    }
}
```

Access Modifiers and Property Accessors

Access modifiers control the visibility and accessibility of class members, while property accessors define how data can be read and written.

1	public Accessible everywhere
2	private Accessible only within the containing class
3	protected Accessible within the class and derived classes
4	internal Accessible within the same assembly

Accessors

- **get** (read)
- **set** (write)
- **init** (write-only during initialization)

```
public class Employee
{
    // Auto-implemented
    public int Id { get; set; }

    // Init-only: can be set at object creation, then immutable
    public string Department { get; init; } = string.Empty;

    // Full property with validation
    private string _email = ;

    public string Email
    {
        get => _email;
        set
        {
            if (!value.Contains("@"))
                throw new FormatException("Invalid email.");
            _email = value;
        }
    }
}

// Usage
var emp = new Employee { Id = 1, Department = "IT" };
emp.Email = "john@example.com";
```

Auto-implemented vs. Full Properties

Auto-implemented Properties

Compiler creates the hidden backing field automatically. Best for simple data transfer with no additional logic.

```
public class Product
{
    public string Name { get; set; }
    public double Price { get; set; }
}
```

Full Properties

You declare the backing field explicitly to add custom logic such as validation, computed values, or change notifications.

```
public class ProductFull
{
    private double _price;
    public double Price
    {
        get => _price;
        set
        {
            if (value < 0)
                throw new ArgumentException("Price cannot be negative.");
            _price = value;
        }
    }
}
```

Data Validation and Computed Properties



Validation

Performed in the set accessor to enforce business rules and data integrity



Computed Properties

Read-only properties that calculate a value dynamically from other fields. No backing field needed

```
public class Rectangle
{
    public double Width { get; set; }
    public double Height { get; set; }

    // Computed property
    public double Area => Width * Height;
    private double _width;
    public double ValidatedWidth
    {
        get => _width;
        set
        {
            if (value <= 0)
                throw new ArgumentException("Width must be > 0");
            _width = value;
        }
    }
}
```

Inheritance: Types, Pros/Cons, and Examples

Inheritance is a mechanism where a new class (derived) acquires properties and behavior of an existing class (base).

Types in C#

- Single inheritance only (a class can only inherit from one base class)
- Supports multilevel chains ($A \rightarrow B \rightarrow C$)
- Hierarchical trees ($A \rightarrow B$ and $A \rightarrow C$)

Pros

- Code reuse
- Logical hierarchy
- Polymorphism

Cons

- Tight coupling
- Fragile base class problem
- Breaks encapsulation if not careful

```
public class Vehicle // Base
{
    public string Brand { get; set; } = string.Empty;
    public void Start() => Console.WriteLine("Vehicle starting...");
}

public class Car : Vehicle // Derived
{
    public int TrunkCapacity { get; set; }
}

// Usage
Car c = new Car();
c.Brand = "Toyota"; // inherited
c.Start();          // inherited method
```

Base vs. Derived Classes & base Keyword

01

Base Class

The parent providing common functionality

02

Derived Class

The child extending or specializing the parent

03

base Keyword

Used to call a base class constructor or access base class members that are hidden or overridden

```
public class Person
{
    public string Name { get; set; }
    public Person(string name) => Name = name;
    public virtual void Introduce() => Console.WriteLine($"I am {Name}.");
}
```

```
public class Student : Person
{
    public string Major { get; set; }

    // Calling base constructor
    public Student(string name, string major) : base(name)
    {
        Major = major;
    }
}
```

```
public override void Introduce()
{
    base.Introduce(); // call base implementation
    Console.WriteLine($"I study {Major}.");
}
}
```

```
// Usage
Student s = new Student("Rifat", "Data Science");
s.Introduce();
```


Method Hiding vs. Overriding

Overriding (override)

Replaces a virtual base method. The derived method is invoked even when accessed through a base class reference (runtime polymorphism).

Hiding (new)

Creates a completely new method with the same name. The base method is still called when accessed through a base class reference.

```
public class Animal
{
    public virtual void Speak() => Console.WriteLine("Animal speaks");
    public void Move() => Console.WriteLine("Animal moves");
}

public class Dog : Animal
{
    // Overriding
    public override void Speak() => Console.WriteLine("Dog barks");

    // Hiding
    public new void Move() => Console.WriteLine("Dog runs");
}

// Demonstration
Animal a = new Dog();
a.Speak(); // "Dog barks" (override works polymorphically)
a.Move(); // "Animal moves" (new hides it; base method called via base ref)
```


Protected Members

The **protected** access modifier makes members accessible within their own class and in derived classes, but not from unrelated classes. This is useful for providing helper methods or internal state to subclasses without exposing them publicly.

```
public class Account
{
    protected decimal Balance { get; set; }

    public Account(decimal initialBalance)
    {
        Balance = initialBalance;
    }

    protected void LogTransaction(string msg) => Console.WriteLine($"LOG: {msg}");
}

public class SavingsAccount : Account
{
    public SavingsAccount(decimal initialBalance) : base(initialBalance) { }

    public void AddInterest(decimal rate)
    {
        // Can access protected Balance and LogTransaction
        decimal interest = Balance * rate;
        Balance += interest;
        LogTransaction($"Interest added: {interest}");
    }
}

// Usage
SavingsAccount s = new SavingsAccount(1000);
s.AddInterest(0.05m);
```

Inheritance vs. Composition

Inheritance

An "IS-A" relationship. Use when the derived class truly is a type of the base class.

Composition

A "HAS-A" relationship. Use when a class needs to use functionality of another class without being that type.

When Composition Wins

- There is no true "IS-A" relationship
- You need to reuse behavior from multiple sources (C# doesn't allow multiple class inheritance)
- You want looser coupling and easier testing

BAD: Inheritance

Penguin IS NOT really a FlyingBird

```
public class Bird
{
    public virtual void Fly() { }
}

public class Penguin : Bird
{
    public override void Fly() => throw new Exception("Can't fly");
}
```

GOOD: Composition

Car HAS-A Engine

```
public class Engine
{
    public void Start() => Console.WriteLine("Engine started");
}

public class Car
{
    private readonly Engine _engine = new Engine(); // Composition
    public void Start()
    {
        _engine.Start();
        Console.WriteLine("Car is ready to go.");
    }
}
```